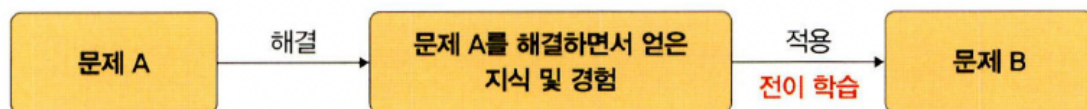


4주차

5.3 전이 학습

전이 학습(Transfer Learning)이란?

- 딥러닝 모델을 제대로 학습하려면 많은 양의 데이터가 필요하지만, 현실적으로 큰 데이터셋을 확보하기 어렵다.
- 이를 해결하기 위해 전이 학습을 사용.
- 전이 학습: 아주 큰 데이터셋(예: ImageNet)으로 미리 학습한 모델의 가중치를 가져와 새로운 과제에 맞게 일부만 수정하여 사용하는 방법.
- 효과: 적은 양의 데이터로도 원하는 문제를 해결 가능.



전이 학습의 방식

전이 학습 두 가지 방법:

1. 특성 추출 (Feature Extraction)

- 사전 학습된 모델의 합성곱 층(convolution layers)은 그대로 사용 (가중치 고정).
- 마지막 완전연결층(fully connected layer)만 새롭게 만들어 학습.
- 즉, 이미지에서 특징을 뽑는 부분은 그대로 쓰고, 분류기만 새로 학습.

2. 미세 조정 (Fine-Tuning)

- 합성곱 층 일부까지 학습을 허용하여, 새로운 데이터에 맞게 가중치를 조금씩 조정.

5.3.1 특성 추출 기법 (Feature Extraction)

구성 요소

1. 합성곱층(Feature Extractor)

- Convolution + Pooling layer로 이루어짐.
- 이미지에서 특징(엣지, 패턴 등)을 추출.

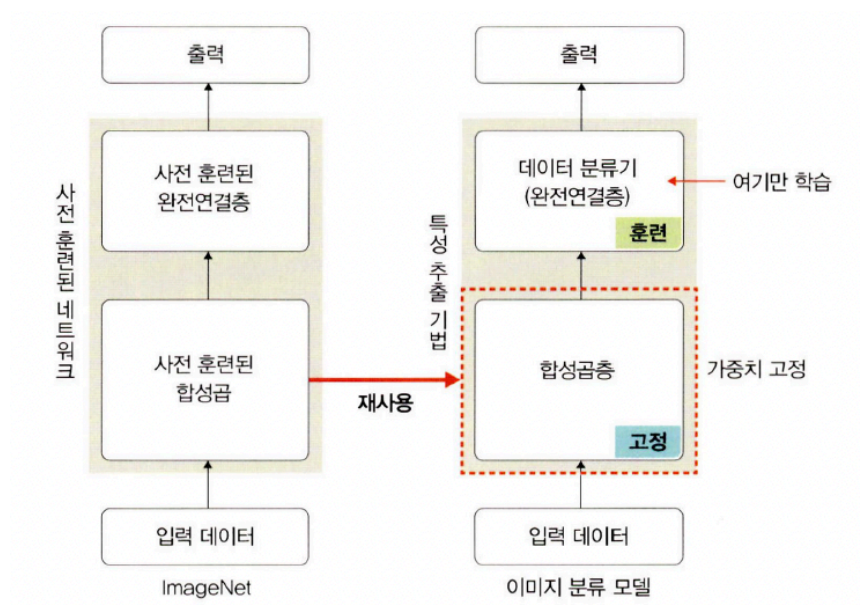
2. 데이터 분류기(Classifier, Fully Connected Layer)

- 추출된 특징을 입력받아 최종적으로 이미지의 클래스를 결정.
- 훈련 방식
 - 합성곱층은 가중치 고정(Freeze) → 학습하지 않음.
 - 데이터 분류기(완전연결층)만 학습.

사용 가능한 사전 학습 모델들

- Xception
- Inception V3
- ResNet50
- VGG16
- VGG19
- MobileNet

그림으로 보는 특성 추출 기법



- 왼쪽 (기존 사전 학습 네트워크)
 - ImageNet 데이터로 학습된 모델 전체 구조 (합성곱층 + 완전연결층).

- 오른쪽 (특성 추출 기법)
 - 합성곱층은 재사용 & 고정.
 - 완전연결층(분류기)만 새로 학습.
 - 따라서, “여기만 학습” (Classifier 부분만 학습).

라이브러리 설치 및 호출

- OpenCV (cv2): 오픈소스 컴퓨터 비전 라이브러리. 이미지 처리에 널리 사용.
- PyTorch & Torchvision: 딥러닝 프레임워크와 시각처리 전용 패키지.
 - torchvision.transforms: 이미지 전처리/데이터 증강 기능.
 - torchvision.models: 다양한 사전 학습된 모델 제공.
 - DataLoader: 데이터를 배치 단위로 효율적으로 불러오는 클래스.
- matplotlib: 시각화 라이브러리.

전처리 과정 (Transform)

전처리 = 이미지 데이터를 신경망에 맞게 변환하는 과정

```
transform = transforms.Compose([
    transforms.Resize([256, 256]),    # 크기 변경
    transforms.RandomResizedCrop(224), # 랜덤 크롭 + 리사이즈
    transforms.RandomHorizontalFlip(), # 랜덤 좌우 반전
    transforms.ToTensor()             # Tensor로 변환
])
```

- Resize: 이미지 크기 통일 (256×256).
- RandomResizedCrop: 이미지를 무작위로 잘라서 224×224 크기로 변환 → 데이터 다양성 확보.
- RandomHorizontalFlip: 이미지를 랜덤으로 좌우 반전 → 모델이 방향에 덜 민감하도록 학습.
- ToTensor: 이미지를 PyTorch Tensor로 변환.

데이터셋 불러오기 (ImageFolder)

```
train_dataset = torchvision.datasets.ImageFolder(
    data_path,
```

```
transform=transform
)
```

- ImageFolder: 폴더 구조에 따라 이미지를 불러옴.
- transform: 불러온 이미지를 전처리 파이프라인에 적용.

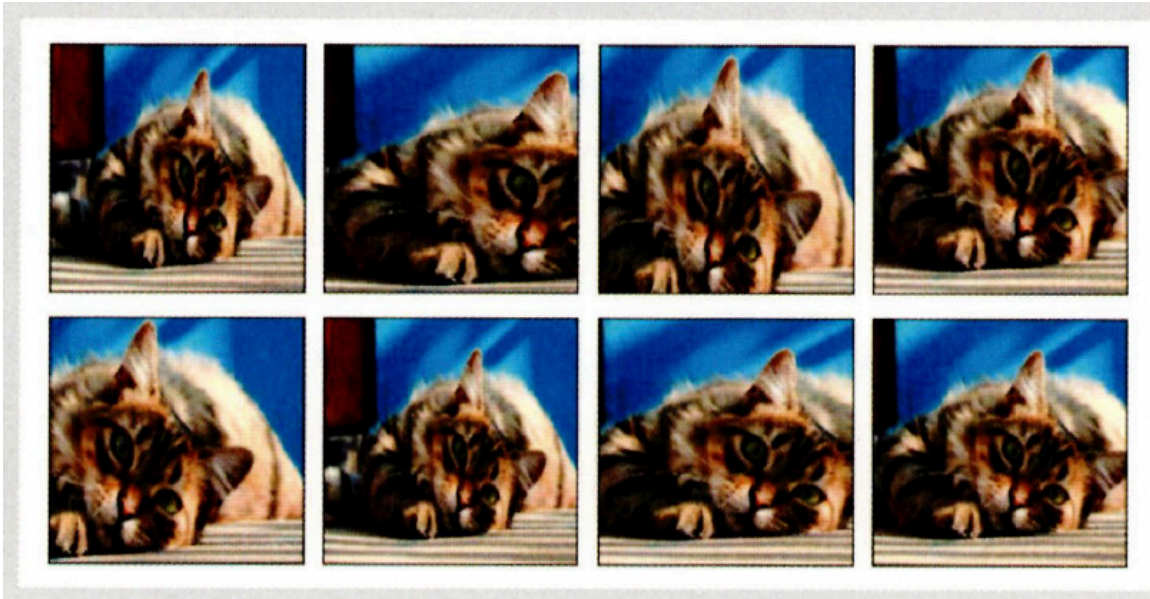
데이터 로더 (DataLoader)

```
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
)
```

- batch_size=32: 한 번 학습할 때 32장의 이미지 사용.
- num_workers=8: 데이터를 불러올 때 병렬 프로세스 8개 사용.
- shuffle=True: 데이터를 매 epoch마다 랜덤하게 섞음 → 일반화 성능 향상.

Data Augmentation

- 목적: 데이터 부족 문제를 해결하고, 모델이 더 일반화된 특징을 학습하도록 도움.
- 방법: 원본 이미지를 변형하여 새로운 학습 샘플 생성.
 - 회전, 자르기, 반전, 색상 변화, 노이즈 추가 등.
- 예시: RandomResizedCrop
 - size=(200,200): 최종 출력 크기
 - scale=(0.1,1): 원본의 10%~100% 크기 중 랜덤
 - ratio=(0.5,2): 가로/세로 비율을 0.5~2 사이에서 랜덤 선택
 - 다양한 비율과 크기로 이미지를 잘라내어 학습 데이터 확장.



- 원본 고양이 이미지를 RandomResizedCrop으로 여러 번 잘라내면, 서로 다른 크기와 비율의 다양한 고양이 이미지가 생성됨.
- 같은 고양이지만 조금씩 다른 위치·크기로 잘린 이미지들이 나옴 → 데이터 다양성 확보.

학습에 사용될 이미지 출력

- 배치 데이터 불러오기

```
samples, labels = iter(train_loader).next()
```

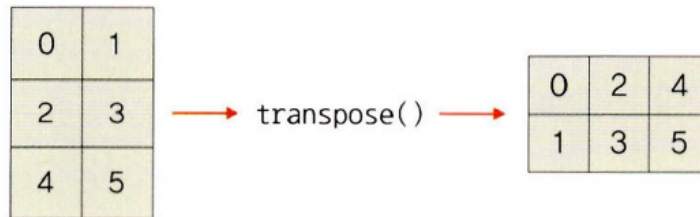
- `iter(train_loader)` : DataLoader를 반복자로 변환.
- `next()` : 그 반복자에서 한 배치(batch) 데이터를 꺼냄.
 - `samples` → 이미지 데이터 (Tensor)
 - `labels` → 각 이미지의 정답(class)

- 시각화 코드

```
a.imshow(np.transpose(samples[i].numpy(), (1, 2, 0)))
```

- PyTorch 이미지 Tensor: (C, H, W) = (채널, 높이, 너비)
- Matplotlib 요구 형식: (H, W, C) = (높이, 너비, 채널)
- 따라서 `np.transpose(..., (1, 2, 0))`로 차원 순서를 바꿔줘야 함.

- `transpose()` 개념
 - `transpose` = 행과 열(또는 차원)을 바꿔주는 연산.
 - 예: $(3, 224, 224) \rightarrow (224, 224, 3)$
 - 이를 하지 않으면 Matplotlib에서 에러 발생:
 - Invalid shape (3, 224, 224) for image data

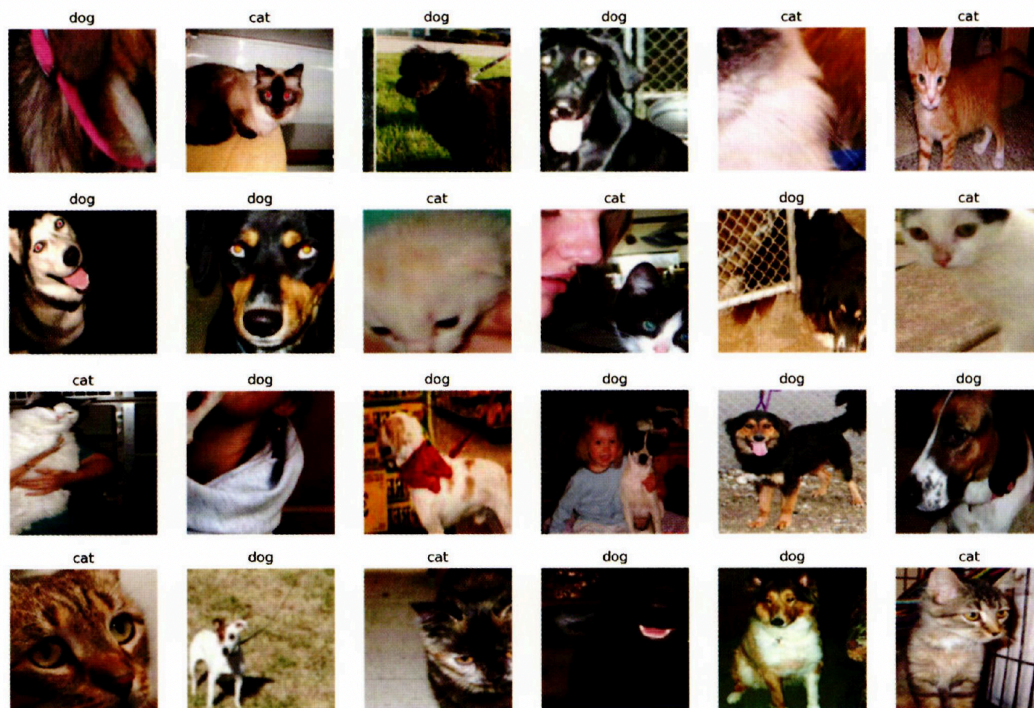


- 행렬 연산과의 연관
 - 두 행렬의 곱을 계산하려면 차원이 맞아야 함.
 - 차원이 맞지 않을 때 `transpose`로 조정 가능.
 - 예:

```
exam = np.arange(24).reshape(2,3,4)
exam.shape # (2, 3, 4)

np.transpose(exam, (2, 1, 0)).shape
# (4, 3, 2)
```

- 실제 데이터 차원이 원하는 순서로 바뀌는 것 확인 가능
- 최종 출력



- 24장의 샘플 이미지(cat, dog 데이터셋)를 4행 × 6열 격자로 출력.
- 각 이미지 위에 라벨(cat 또는 dog)을 제목으로 표시.

사전 학습된 모델 내려받기

```
resnet18 = models.resnet18(pretrained=True)
```

- pretrained=True : ImageNet 데이터셋으로 학습된 가중치를 불러옴.
- 장점: 처음부터 학습할 필요 없이 빠르게 좋은 성능을 낼 수 있음.
- 파이토치에서 무작위의 가중치로 다양한 모델 구성 가능

```

import torchvision.models as models
resnet18 = models.resnet18()
alexnet = models.alexnet()
vgg16 = models.vgg16()
squeezenet = models.squeezenet1_0()
densenet = models.densenet161()
inception = models.inception_v3()
googlenet = models.googlenet()
shufflenet = models.shufflenet_v2_x1_0()
mobilenet_v2 = models.mobilenet_v2()
mobilenet_v3_large = models.mobilenet_v3_large()
mobilenet_v3_small = models.mobilenet_v3_small()
resnext50_32x4d = models.resnext50_32x4d()
wide_resnet50_2 = models.wide_resnet50_2()
mnasnet = models.mnasnet1_0()

```

사전 훈련된 모델의 파라미터 학습 유무 지정

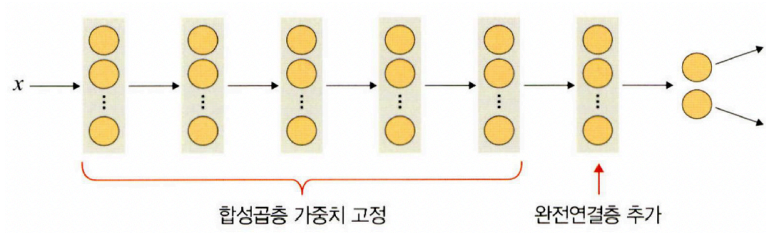
```

def set_parameter_requires_grad(model, feature_extracting=True):
    if feature_extracting:
        for param in model.parameters():
            param.requires_grad = False

```

- requires_grad=False : 역전파 시 가중치를 업데이트하지 않음 → 합성곱층 (Convolution Layer)을 고정.
- CNN의 합성곱층은 일반적인 이미지 특징(모서리, 질감 등)을 이미 잘 학습했기 때문에, 그대로 사용 가능.
- 따라서 새로운 데이터셋(예: 개/고양이 분류)에서는 마지막 완전연결층(FC Layer) 만 새로 학습.

ResNet18에 완전연결층 추가



```
resnet18.fc = nn.Linear(512, 2)
```

- 기존의 1000개 클래스 분류 → 2개 클래스(개, 고양이) 로 변경.
- 512 = 마지막 합성곱층의 출력 크기, 2 = 분류할 클래스 수.

모델의 파라미터값 확인

```
for name, param in resnet18.named_parameters():
    if param.requires_grad:
        print(name)
```

- requires_grad=True 인 파라미터만 학습 대상.
- 즉, 새로 추가한 FC Layer의 weight와 bias만 학습됨.

모델 객체 생성 및 손실 함수 정의

```
model = models.resnet18(pretrained=True)

# 합성곱층 고정
for param in model.parameters():
    param.requires_grad = False

# 마지막 층 교체
model.fc = nn.Linear(512, 2)

# 마지막 층만 학습
for param in model.fc.parameters():
    param.requires_grad = True

# 옵티마이저와 손실 함수
```

```
optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss()
```

- Optimizer: Adam → FC Layer 파라미터만 업데이트.
- Loss: CrossEntropyLoss → 분류 문제에 적합.

최종 모델 구조

- Conv2D, BatchNorm, ReLU, Residual Block 들이 꼭 이어짐.

```
(layer1): Sequential(
  (0): BasicBlock(
    (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  )
)
```

- 마지막으로

```
(avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
(fc): Linear(in_features=512, out_features=2, bias=True)
```

- 개/고양이 2클래스 분류용 모델로 완성.

모델 학습을 위한 함수 생성

- train_model() 함수
 - 모델을 학습시키기 위한 전체 흐름을 정의한 함수.
 - 입력값:
 - model (네트워크),
 - dataloaders (데이터 배치),
 - criterion (손실 함수),
 - optimizer (최적화 방법),
 - device (CPU or GPU),

- num_epochs (학습 횟수, 기본 13번).
- 추적 변수:
 - acc_history → epoch별 정확도 저장
 - loss_history → epoch별 손실 저장
 - best_acc → 최고 정확도 기록
- Epoch 학습 과정
 - 각 epoch마다 다음을 반복:
 1. 데이터 로드
 - for inputs, labels in dataloaders
 - 데이터를 배치 단위로 GPU/CPU에 전달.
 2. 순전파 (Forward)
 - outputs = model(inputs)
 - 모델이 예측값 산출.
 3. 손실 계산 (Loss)
 - loss = criterion(outputs, labels)
 - 예측과 정답 비교.
 4. 역전파 (Backward)
 - loss.backward() → 그래디언트 계산
 - optimizer.step() → 파라미터 업데이트
 5. 평가
 - running_loss → 손실 합산
 - running_corrects → 맞춘 개수 합산
 6. Epoch 결과 계산
 - epoch_loss = 평균 손실
 - epoch_acc = 평균 정확도

파라미터 학습 결과를 옵티마이저에 전달

- ResNet18의 합성곱층(convolution layer) 은 고정 (학습 X).

- 마지막 완전연결층(fc) 만 학습 → 개/고양이 분류.
- params_to_update 리스트에 requires_grad=True인 파라미터만 전달.
- optimizer = Adam(params_to_update) 로 최적화.

모델 학습

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()
train_acc_hist, train_loss_hist = train_model(resnet18, train_loader, criterion,
optimizer, device)
```

- GPU가 가능하면 CUDA 사용, 아니면 CPU.
- 손실 함수: CrossEntropyLoss (분류 문제에 적합).
- train_model 실행으로 학습 시작.
- 결과:

```
Epoch 0/12
-----
Loss: 0.5614 Acc: 0.7221

Epoch 1/12
-----
Loss: 0.4387 Acc: 0.7948

Epoch 2/12
-----
Loss: 0.3055 Acc: 0.8961

Epoch 12/12
-----
Loss: 0.1985 Acc: 0.9169

Training complete in 6m 27s
Best Acc: 0.937662
```

- 최고 정확도 Best Acc = 93.7%

- 모델이 점점 손실을 줄이고 정확도를 높여 성공적으로 학습한 것 확인

테스트 데이터 호출 및 전처리

```
transform = transforms.Compose([
    transforms.Resize(224),
    transforms.CenterCrop(224),
    transforms.ToTensor()
])
test_dataset = torchvision.datasets.ImageFolder(root=test_path, transform=transform)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=32, shuffle=True)
```

- transform: 테스트 이미지 크기를 맞추고 텐서로 변환.
- ImageFolder: 지정된 폴더(test_path)에 있는 이미지를 불러옴.
- DataLoader: 배치 단위(32장)로 데이터를 나누어 모델에 전달할 수 있도록 함.

테스트 데이터 평가 함수 생성

```
def eval_model(model, dataloaders, device):
    saved_models = glob.glob('./chap05/data/catanddog/*.pth')
    for model_path in saved_models:
        model.load_state_dict(torch.load(model_path))
        model.eval()
    ...
```

- glob: 특정 폴더에서 .pth 파일들(학습된 모델 파라미터)을 모두 불러옴.
- model.load_state_dict(): 저장된 모델 파라미터 로드.
- model.eval(): 평가 모드로 전환 (드롭아웃, 배치정규화 등 학습용 동작을 끄).
- torch.no_grad(): 평가 시에는 gradient 계산을 하지 않아 메모리와 속도 절약.
- 정확도 계산

```
_, preds = torch.max(outputs.data, 1)
running_corrects += preds.eq(labels.data).sum()
```

```
epoch_acc = running_corrects.double() / len(dataloaders.dataset)
```

- `torch.max(outputs, 1)`: 모델 출력값에서 가장 큰 값의 인덱스를 클래스 예측으로 사용.
- `preds.eq(labels)`: 예측(preds)과 실제 정답(labels)이 같은지 비교.
- 정확도(acc): 전체 데이터 중 맞춘 비율.

테스트 데이터를 평가 함수에 적용

```
val_acc_hist = eval_model(resnet18, test_loader, device)
```

- 여러 개의 저장된 모델을 불러와서 각각 테스트 데이터셋에 대해 평가.
- 각 모델마다 정확도가 출력됨.
- 결과:

```
Loading model ../chap05/data/catanddog\11.pth
Acc: 0.9286
```

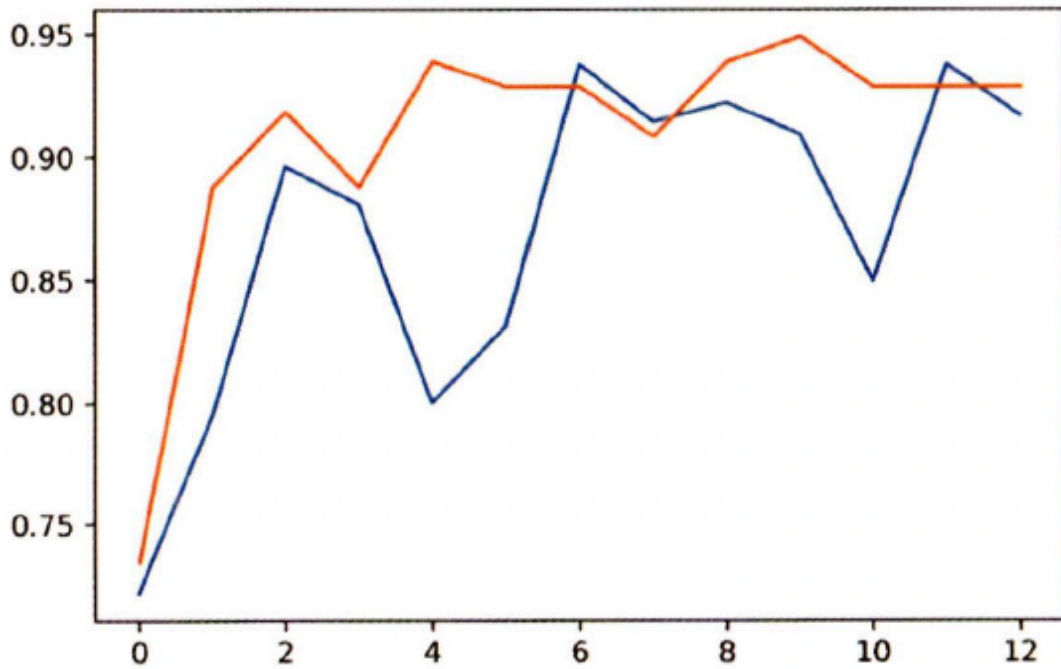
```
Loading model ../chap05/data/catanddog\12.pth
Acc: 0.9286
```

```
Validation complete in 1m 52s
Best Acc: 0.948980
```

- 테스트 데이터 정확도 최대 약 95% 달성.
- 사전학습된 모델(ResNet18)을 활용했기 때문에 높은 성능을 빠르게 얻음.
- 만약 모델을 처음부터 직접 설계하고 학습했다면 훨씬 오래 걸렸을 것.

훈련과 테스트 데이터의 정확도를 그래프로 확인

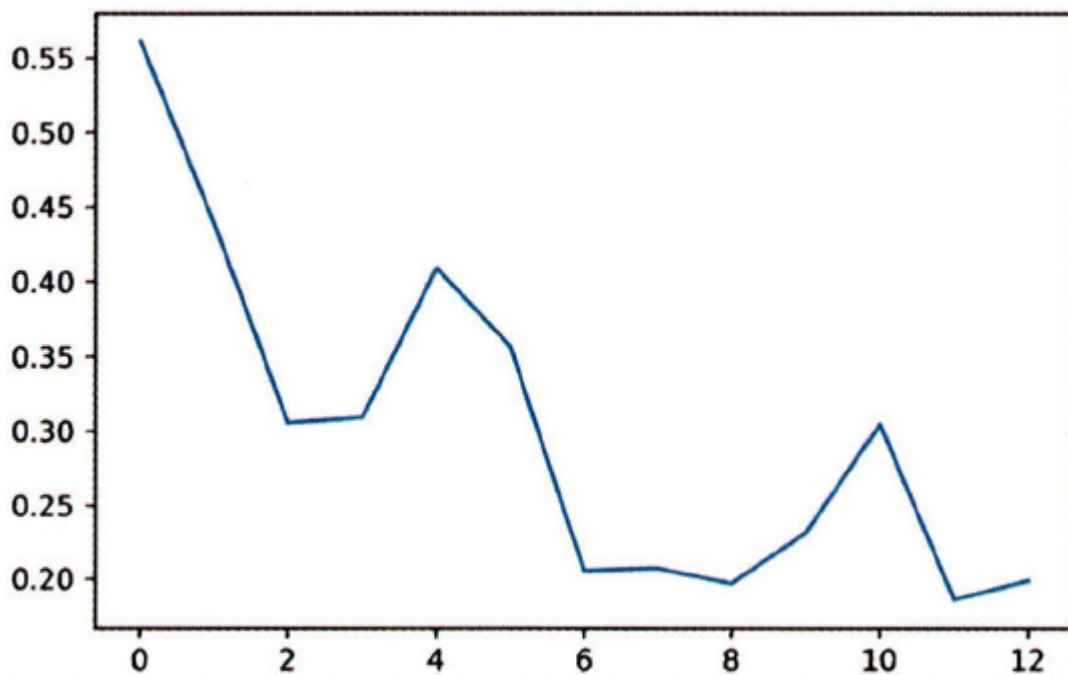
- `plt.plot(train_acc_hist)` → 훈련 데이터 정확도
- `plt.plot(val_acc_hist)` → 검증(테스트) 데이터 정확도



- Epoch이 진행될수록 두 정확도 모두 상승 → 학습이 잘 진행되고 있음
- 12 Epoch 기준, 약 94% 정확도 도달

훈련 데이터의 오차에 대한 그래프 확인

- `plt.plot(train_loss_hist)` → 훈련 데이터 손실



- Epoch이 진행될수록 Loss가 감소 → 모델이 데이터를 잘 학습하고 있다는 의미
- Loss 감소 + Accuracy 증가 = 올바른 학습 패턴

예측 이미지 출력을 위한 전처리 함수

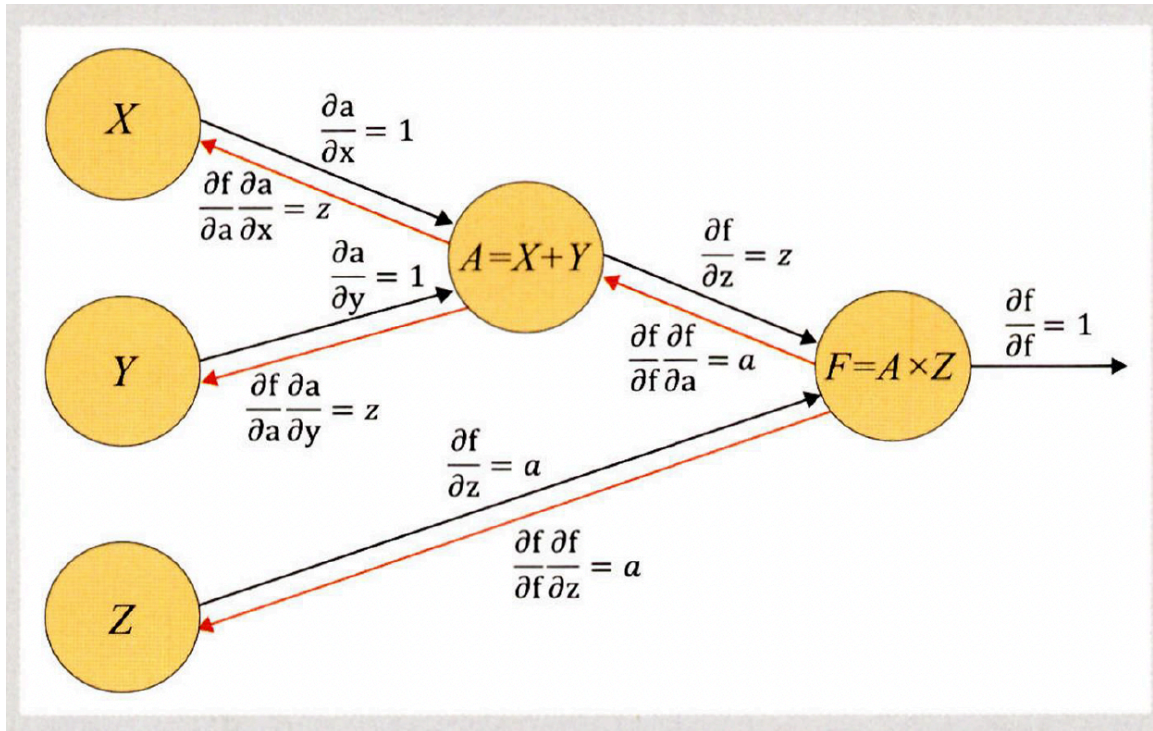
```
def im_convert(tensor):
    image = tensor.clone().detach().numpy()
    image = image.transpose(1, 2, 0)  # (C, H, W) → (H, W, C)
    image = image * 0.5 + 0.5         # 정규화 복원
    image = image.clip(0, 1)         # 값 범위 제한 (0~1)
    return image
```

- 학습된 Tensor 이미지를 사람이 볼 수 있는 이미지로 변환
- tensor.clone(), tensor.detach(), tensor.clone.detach() 차이

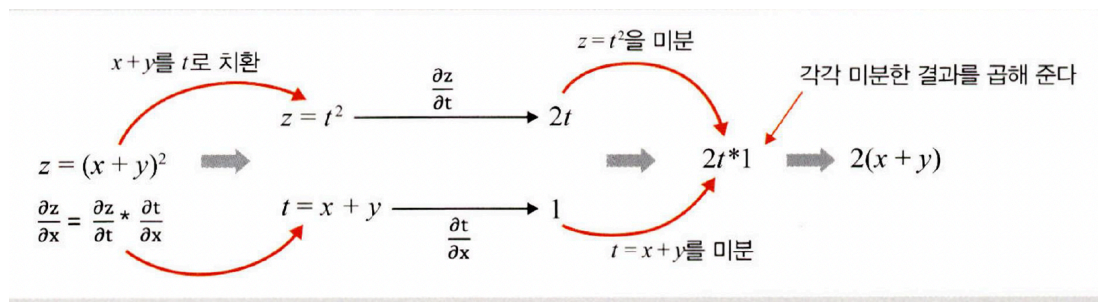
구분	메모리	계산 그래프 상주 유무
tensor.clone()	새롭게 할당	계산 그래프에 계속 상주
tensor.detach()	공유해서 사용	계산 그래프에 상주하지 않음
tensor.clone().detach()	새롭게 할당	계산 그래프에 상주하지 않음

계산 그래프 과 연쇄 법칙

- 계산 그래프 (Computational Graph): 연산 과정을 시각적으로 표현 (노드=연산, 엣지=데이터 흐름)



- 연쇄 법칙 (Chain Rule): 복잡한 함수 미분을 단계별로 나누어 계산
 - 합성 함수의 미분



- 역전파(backpropagation)의 핵심 원리

개와 고양이 예측 결과 출력

```
classes = {0: 'cat', 1: 'dog'}
images, labels = next(iter(test_loader))
outputs = model(images)
_, preds = torch.max(outputs, 1)
```

- 고양이/개 분류 결과 출력
- Subplot

- `add_subplot`
 - 한 화면(Figure) 안에 여러 개의 이미지를 배치할 때 사용.
 - 파라미터:
 - 첫 번째 (행 개수) → 예: 2 → 2줄에 배치.
 - 두 번째 (열 개수) → 예: 10 → 한 줄에 10개 이미지.
 - 세 번째 (인덱스) → `idx+1` → 현재 몇 번째 subplot인지 지정.
 - `xticks=[], yticks=[]` → X축, Y축 눈금을 제거.
- `imshow`
 - 이미지를 subplot에 출력.
- `set_title`
 - subplot 위에 제목 표시.
 - 예측 결과(preds)와 실제 정답(labels)을 함께 표시.
 - 색상 지정:
 - 정답과 예측이 같으면 초록색(green).
 - 다르면 빨간색(red).
- `plt.subplots_adjust`
 - Figure 내 subplot 배치를 조정.
 - 파라미터:
 - left, bottom, right, top → 전체 Figure 안에서 subplot 위치.
 - hspace, wspace → subplot 간 간격(세로, 가로).
- 결과



- 전체 정확도는 높지만, 일부 고양이/개 이미지에서 예측이 틀림
- 개선 방법:

1. 데이터 양 증가
2. Epoch 더 늘려 학습
3. 데이터 증강(Data Augmentation) 적용

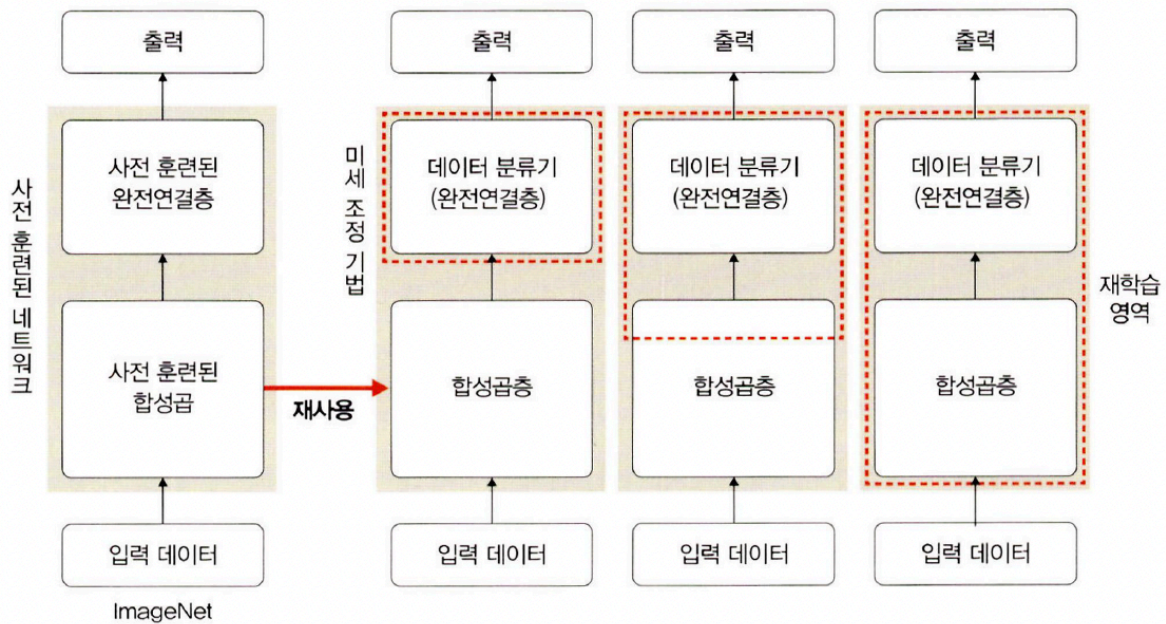
5.3.2 미세 조정 기법

개념

- 사전 훈련된(pre-trained) 모델을 새로운 데이터셋에 맞게 조금 수정해서 학습시키는 방법.
- 특징 추출 기반 전이학습보다 더 깊게, 모델의 가중치 일부(또는 전체)를 업데이트하여 성능을 향상.
- 특히, 기존 데이터셋과 새 데이터셋의 특성이 다를 때 효과적.

전략 (데이터셋 크기 & 유사성에 따른 방법)

1. 데이터셋 크고 + 기존 모델과 유사성이 큼
 - 모델 전체 재학습 (all layers fine-tuning)
 - 데이터가 충분하고, 새 데이터가 기존 모델과 유사하기 때문에 전체 학습이 가능.
2. 데이터셋 크고 + 기존 모델과 유사성이 작음
 - 뒷부분(FC layer + 일부 합성곱 층)만 재학습
 - 일반적인 저수준 특징(엣지, 모양 등)은 유지하고, 고수준 특징만 새로 학습.
3. 데이터셋 작고 + 기존 모델과 유사성이 작음
 - 일부 층만 제한적으로 재학습
 - 데이터 부족으로 오버피팅 위험 → 네트워크 일부(FC 등)만 학습.
4. 데이터셋 작고 + 기존 모델과 유사성이 큼
 - 마지막 분류기(FC layer)만 학습
 - 작은 데이터셋이라 전체 fine-tuning 하면 과적합 위험 → 분류기만 조정.



5.4 설명 가능한 CNN

설명 가능한 CNN (Explainable CNN)이란?



- CNN은 입력 → 합성곱/풀링 계층 → 분류 계층 과정을 거쳐 결과를 출력.
- 하지만 내부에서 어떤 특징(feature)을 추출해서 최종 예측을 내리는지 사람이 이해하기 어려움 → 블랙박스 문제.
- 따라서 CNN을 신뢰하기 위해서는 CNN의 처리 과정을 시각화/설명할 필요가 있음.

특성 맵 시각화

- 특성 맵(feature map, activation map): 합성곱 계층에서 추출된 이미지 특징을 보여주는 맵.
- 특징 맵 시각화는 "입력 이미지에서 어떤 부분을 네트워크가 집중해서 보고 있는가"를 보여줌.
- 이를 통해 CNN이 올바르게 특징을 추출하고 있는지 확인 가능.

- 사용 라이브러리
 - matplotlib: 시각화
 - PIL: 이미지 처리
 - cv2: OpenCV
 - torch, torchvision: 딥러닝 프레임워크

설명 가능한 네트워크 생성

```
class XAI(torch.nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),  # ----- inplace=True는 기존의 데이터를 연산의
                                   # 결과값으로 대체하는 것을 의미
            nn.Dropout(0.3),
            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
```

```
def forward(self, x):
    x = self.features(x)
    x = x.view(-1, 512)
    x = self.classifier(x)
    return F.log_softmax(x)
```

- 합성곱 계층(Conv + ReLU + BatchNorm + Dropout + Pooling) 여러 개 + 완전연결층(FC Layer)
- 활성화 함수: ReLU 사용.
- features: CNN 합성곱 계층 집합.
- classifier: 완전연결층 집합.
- forward(): 입력 → 특징 추출 → Flatten → 분류기 → log_softmax 출력.
- 출력층: log_softmax (소프트맥스 확률값의 log를 취한 값).

- 소프트맥스만 쓰면 기울기 소멸(vanishing gradient) 문제 발생 가능 → 로그 소프트맥스를 사용해 안정적인 학습 유도.
- Softmax 수식

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$

- Log Softmax 수식

$$\begin{aligned} \text{logsoftmax} &= \log\left(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}\right) \\ &= x_i - \log\left(\sum_{j=1} e^{x_j}\right) \end{aligned}$$

모델 객체화

```
model = XAI()      # XAI라는 이름의 객체(모델) 생성
model.to(device)   # 모델을 CPU 또는 GPU 장치에 할당
model.eval()       # 평가 모드로 전환 (테스트 데이터에 사용)
```

- `model = XAI()`
→ XAI 클래스(앞에서 정의한 CNN 구조)를 불러와 모델 객체 생성.
- `model.to(device)`
→ 학습 또는 평가 시 모델을 사용할 장치(CPU / GPU)에 올림.
- `model.eval()`
→ 학습 시 쓰이는 Dropout, BatchNorm 등을 평가 모드로 바꿔, 추론(inference)용으로 고정.
- `model.to(device)` : GPU/CPU 장치 할당.
- `model.eval()` : 평가 모드로 전환.
- 결과:

```
XAI(
    (features): Sequential(
      (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), bias=False)
      (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): ReLU(inplace=True)
      (3): Dropout(p=0.3, inplace=False)
      (4): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (6): ReLU(inplace=True)
```

- 생성된 모델은 합성곱 계층(Conv2d), 정규화(BatchNorm), 활성화(ReLU), 풀링(MaxPool), Dropout 등이 반복적으로 쌓인 구조.

특성 맵을 확인하기 위한 클래스 정의

- 특성 맵 (Feature Map)
 - CNN에서 입력 이미지 + 필터(커널) 연산 결과로 나오는 출력.
 - 즉, 합성곱 계층을 거친 후 이미지의 특징이 반영된 결과 값.
 - 네트워크 내부에서 어떤 특징을 추출했는지 시각적으로 확인 가능.
- Hook
 - PyTorch에서 제공하는 기능으로, 중간 계층의 입력/출력 값을 확인할 수 있음.
 - 매번 print()를 쓰지 않고도 원하는 계층의 값이나 기울기를 가져올 수 있음.
 - 두 종류:
 1. register_forward_hook → 순전파(Forward) 결과 값 확인
 2. register_backward_hook → 역전파(Backward) 시 기울기 확인

```
class LayerActivations:
    features = []

    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn)

    def hook_fn(self, module, input, output):
        self.features = output.detach().numpy()
```

```
def remove(self):
    self.hook.remove()
```

- register_forward_hook : 지정한 layer의 **출력(feature map)**을 가져옴.
- hook_fn : 가져온 출력을 numpy로 변환해서 저장.
- remove() : hook 해제 (메모리 누수 방지).
- 예시:

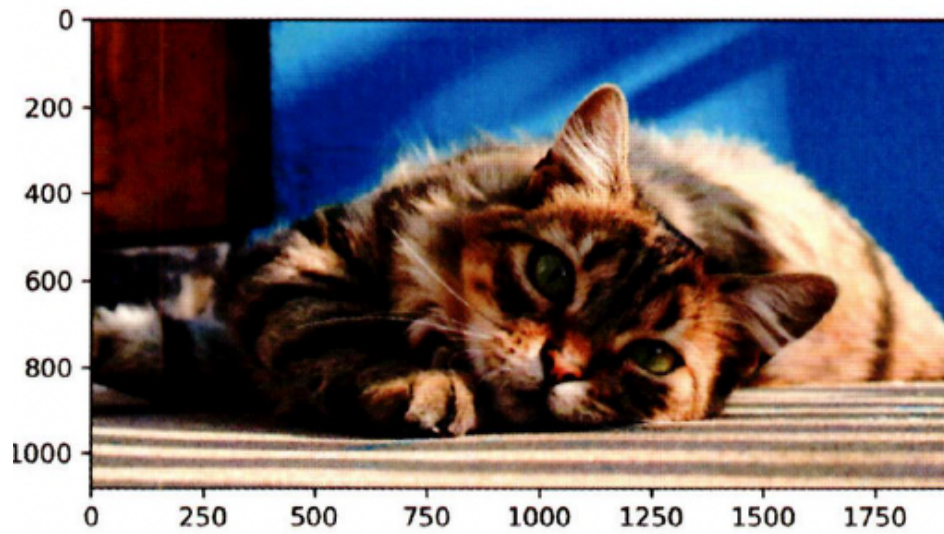
```
x = torch.Tensor([0,1,2,3]).requires_grad_()
y = torch.Tensor([4,5,6,7]).requires_grad_()
w = torch.Tensor([1,2,3,4]).requires_grad_()
z = x + y
o = w.matmul(z)
o.backward()
```

- requires_grad_() → 해당 텐서에 대해 기울기 계산 가능 설정.
- z = x + y → 중간 결과 (Hook을 쓰면 여기 값 확인 가능).
- o.backward() → w, z 등에 대한 기울기 계산.
- Hook을 쓰는 이유
 - 중간 값(z) 같은 경우는 PyTorch가 기본적으로 기울기를 저장하지 않음.
 - z.register_hook(hook_fn)을 사용하면, 중간 변수의 기울기까지 확인 가능.
 - 따라서 학습 과정에서 **특성 맵(중간 출력) + 기울기(역전파 값)**을 직접 확인할 수 있음.

이미지 호출

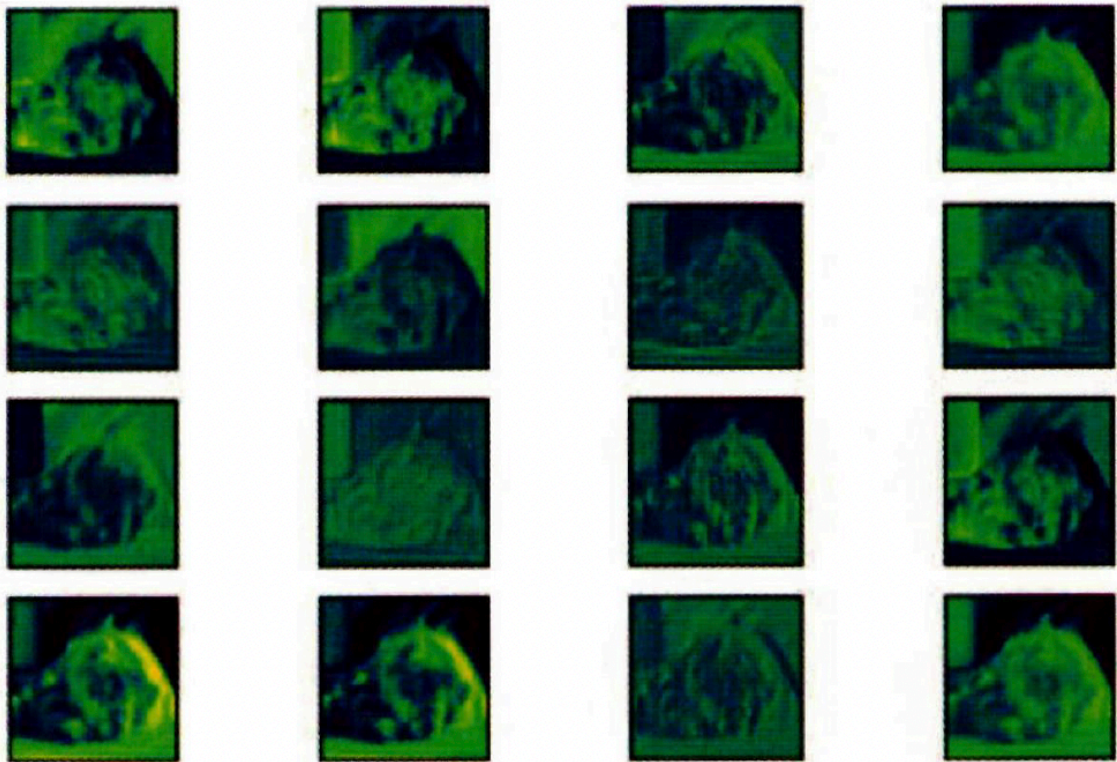
- OpenCV(cv2.imread)로 이미지를 불러온 후, 크기를 cv2.resize로 조정.
- ToTensor()와 unsqueeze(0)을 사용해 PyTorch 텐서로 변환.
 - unsqueeze(0) : 차원을 하나 추가해 (Batch, Channel, Height, Width) 형태로 맞춤.
- Hook 함수로 특성 맵 추출
 - register_forward_hook을 사용해 특정 레이어의 출력을 가져옴.
 - Hook은 중간 계층의 출력(특성 맵)을 확인할 수 있게 해줌.

- LayerActivations 클래스로 hook 등록 및 제거를 관리.

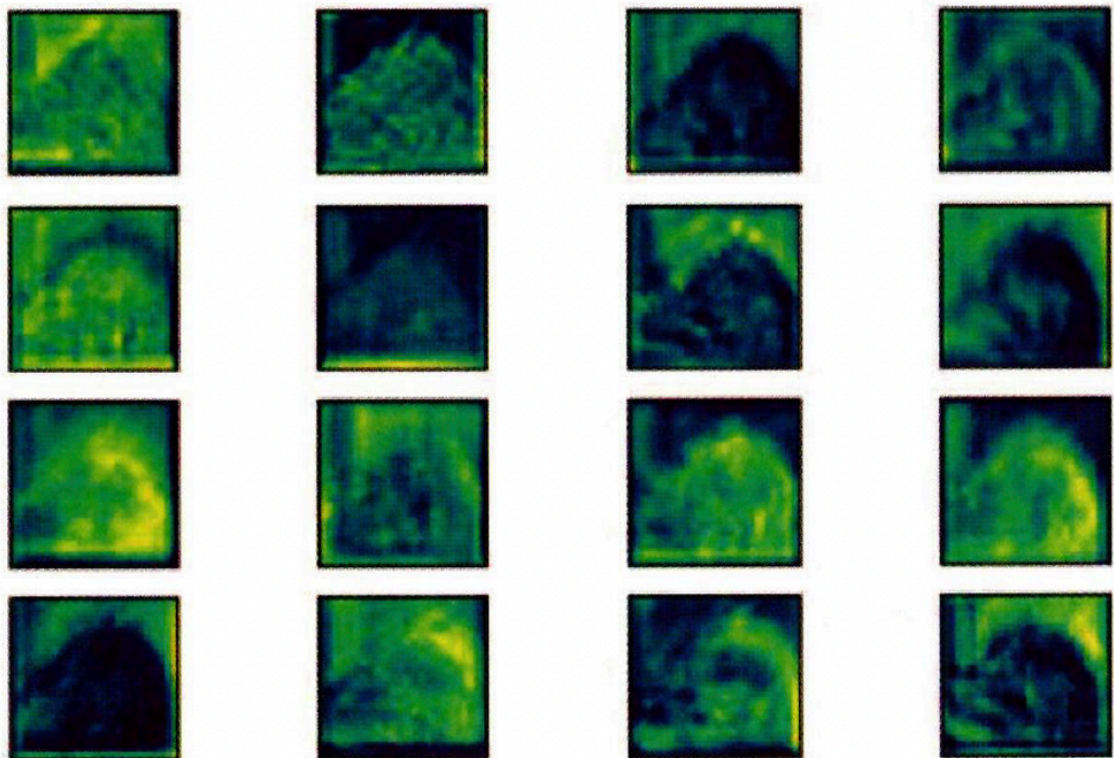


Conv2d 특성 맵 확인 + 특성 맵 시각화

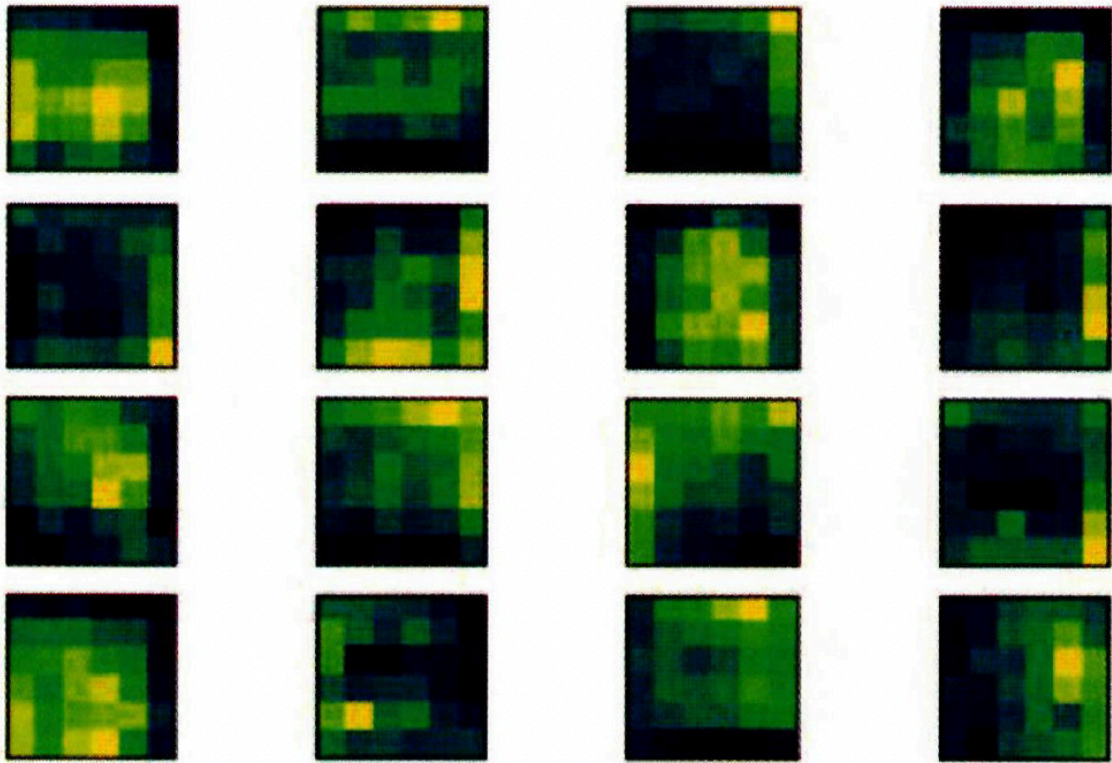
- CNN의 Conv 레이어는 입력 이미지에서 점점 추상적인 특징을 뽑아냄.
- plt.subplots()를 이용해 여러 개의 특성 맵을 그리드 형태로 시각화.
- 레이어가 깊어질수록 이미지의 디테일은 사라지고 분류에 필요한 특징만 강화됨.
- 이를 통해 CNN 내부의 동작을 해석할 수 있음.
- 0번째 Conv 레이어:



- 입력 이미지의 가장자리, 패턴 등이 보임
- 20번째 Conv 레이어:



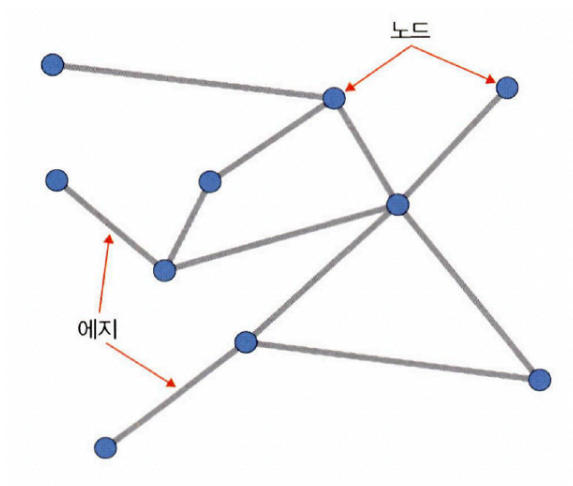
- 형태가 흐릿해지고 중요한 특징(윤곽, 질감 등)만 남음.
- 40번째 Conv 레이어:



- 원래 이미지는 알아볼 수 없고, 학습된 고차원 특징만 남음.

5.5 그래프 합성곱 네트워크

5.5.1. 그래프란?



- 구성요소
 - 노드(Node, Vertex): 점(원), 데이터의 개별 객체를 의미 (예: 사람, 문서, 단백질 등)
 - 엣지(Edge): 두 노드를 연결하는 선, 관계를 의미 (예: 친구 관계, 인용 관계, 단백질 상호작용 등)
- 종류: 방향이 있는 그래프(Directed), 방향이 없는 그래프(Undirected)

5.5.2 그래프 신경망

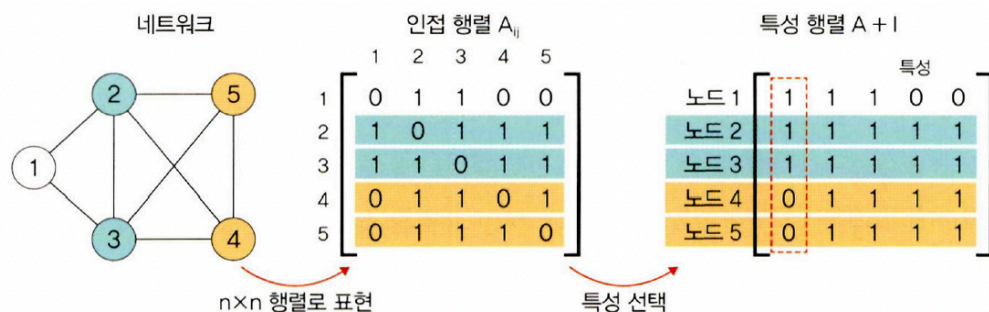
- 그래프 데이터를 처리하기 위한 신경망 구조
- 일반적인 행렬/이미지 데이터가 아니라 노드와 엣지로 표현되는 데이터를 학습하는 데 사용됨
- 그래프 데이터를 수학적으로 표현하기 위해 두 가지 행렬이 사용됨:

1. 인접 행렬(Adjacency Matrix, A)

- n개의 노드가 있을 때 $n \times n$ 크기 행렬로 표현
- 노드 i와 j가 연결되면 $A[i][j] = 1$, 아니면 0
- 그래프 구조 자체를 수학적으로 나타냄

2. 특성 행렬(Feature Matrix, X)

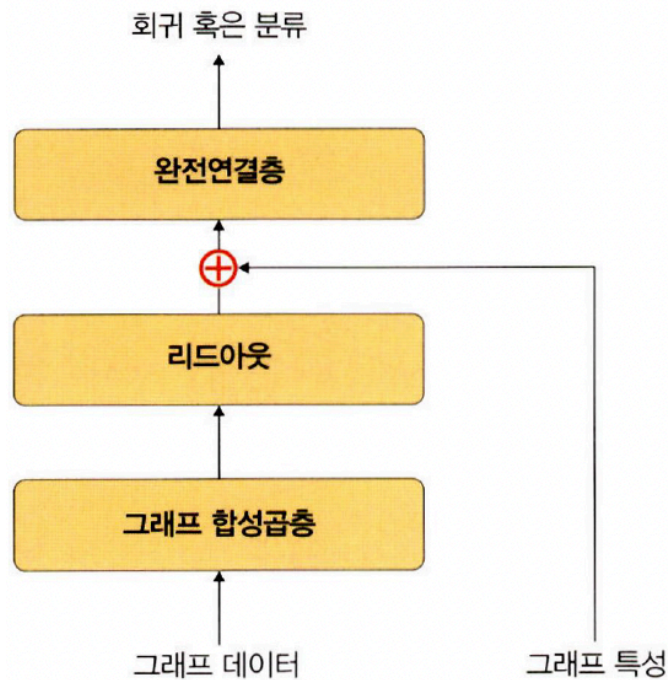
- 각 노드의 속성을 담은 행렬
- 예: 사용자라면 나이, 성별, 관심사 / 논문이라면 단어 벡터, 카테고리 등
- 행(Row) = 노드 / 열(Column) = 해당 노드의 특징



5.5.3. 그래프 합성곱 네트워크

- 핵심 개념:

- CNN이 이미지(격자 구조)에서 인접 픽셀 정보를 합성곱으로 학습하듯, GCN은 그래프에서 인접 노드의 정보를 모아 학습하는 방법임.



- 구조
 1. 그래프 합성곱층 (Graph Convolutional Layer)
 - 각 노드가 자신 + 이웃 노드들의 특징을 합쳐서 새로운 특징을 만들
 - 즉, 인접 행렬 A와 특성 행렬 X를 이용해 새로운 특성 벡터를 생성
 2. 리드아웃(Readout) 층
 - 전체 노드 특징을 모아 하나의 벡터로 요약 (평균, 합 등)
 - 그래프 전체의 표현을 만들어냄
 3. 완전연결층(FC, Fully Connected Layer)
 - 최종적으로 분류(Classification)나 회귀(Regression) 같은 예측 작업 수행
- 활용 예시
 - SNS 관계 네트워크 분석 (예: 친구 추천, 커뮤니티 탐지)
 - 학술 연구 인용 네트워크 (논문 간 연결 관계 분석)
 - 화학 구조 분석 (단백질/분자의 그래프 구조 이해)
 - 3D Mesh 데이터 처리